

Applied Graph Theory

MAT 4409

Contents

List of Symbols	2
1 Algebraic and Structural Properties	3
1.1 Graphs – Review of the Basic Theory	3
1.2 Cartesian Products	3
1.3 Subgraphs	4
1.4 Walks, Paths, and Cycles	4
1.5 Distances and Connectedness	5
1.6 Bipartite Graphs	6
1.7 Trees	6
1.8 Adjacency Matrices	7
1.9 Graph Spectra	8
1.10 Application: Centrality and Clustering	9
1.10.1 Centrality	9
1.10.2 Clustering (Optional)	10
2 Network Reliability and Routing	11
2.1 Cutvertices, Cutedges, and Blocks	11
2.1.1 Application: Network Vulnerability Assessment	13
2.2 Connectivity	13
2.3 Minimum Spanning Trees	14
2.4 All-Pairs Shortest Paths	17
2.5 Application: Routing Tables	18

List of Symbols

$V(G)$	Vertex set of graph G
$E(G)$	Edge set of graph G
$\sim$	is adjacent to
$\deg(v)$	Degree of vertex v
$\delta(G)$	Minimum degree of graph G
$\Delta(G)$	Maximum degree of graph G
$\text{ecc}(v)$	Eccentricity of vertex v
$\text{rad}(G)$	Radius of graph G
$\text{diam}(G)$	Diameter of graph G
$A(G)$	Adjacency matrix of graph G
$G - v$	Graph obtained from G by deleting vertex v and all edges incident with v
$G - e$	Graph obtained from G by deleting edge e
$G + uv$	Graph obtained from G by adding an edge between non-adjacent vertices u and v
K_n	Complete graph of order n (size $\binom{n}{2}$)
P_n	Path graph of order n (size $n - 1$)
C_n	Cycle graph of order n (size n)
Q_k	Hypercube graph of order 2^k

1 Algebraic and Structural Properties

1.1 Graphs – Review of the Basic Theory

We first review basic graph theory. Throughout these notes, all graphs are finite, simple, and undirected unless stated otherwise.

A **graph** is an ordered pair $G = (V, E)$ where V is a non-empty set called the **vertex set**, whose elements are the **vertices** of G , and E is a set of unordered pairs of distinct vertices of G , whose elements are the **edges** of G . The **order** of G is its number of vertices and the **size** of G is its number of edges.

Two vertices u and v of G are **adjacent**, denoted by $u \sim v$, if $uv \in E$ (note that uv here denotes an unordered pair of vertices, and $uv = vu$). If there is no edge between u and v , then they are **nonadjacent**, denoted by $u \not\sim v$. The vertex v and the edge uv are said to be **incident** with each other. The **degree** of a vertex v is the number of edges incident with it, or equivalently, the number of vertices adjacent with it, and is denoted by $\deg(v)$.

Lemma 1.1 (Handshaking Lemma). *The sum of the degrees of all vertices of a graph is twice its size.*

Proof. Let G be a graph. As each edge is incident with exactly two vertices, each edge is counted once by two terms in the sum of the vertex degrees. Thus,

$$\sum_{v \in V(G)} \deg v = 2|E(G)|. \quad \square$$

Corollary 1.2. *The number of vertices of odd degree in a graph is always even.*

Proof. Exercise. $\square$

The **minimum degree** of G is $\delta(G) = \min_{v \in V(G)} \deg v$, and the **maximum degree** of G is $\Delta(G) = \max_{v \in V(G)} \deg v$. A **regular graph** is a graph in which all vertices have the same degree. This common value of the degree is the **regularity** of the graph. A regular graph of regularity k is a **k -regular graph**. A **cubic graph** is a 3-regular graph.

Exercise 1.1. Prove the following:

1. Any cubic graph has even order. For which other regular graphs will this statement hold?
2. In any graph on two or more vertices, at least two vertices have the same degree.
3. If G is a graph of order n and size m , then $\delta(G) \leq \frac{2m}{n} \leq \Delta(G)$.

1.2 Cartesian Products

The **Cartesian product** of two graphs G and H is the graph denoted by $G \times H$, whose vertex set is $V(G) \times V(H)$ (the Cartesian product of the vertex sets $V(G)$ and $V(H)$, consisting of all ordered pairs (u, v) where u is a vertex of G and v is a vertex of H), in which two vertices (u_1, v_1) and (u_2, v_2) are adjacent if and only if either $u_1 = u_2$ and $v_1 \sim v_2$ or $u_1 \sim u_2$ and $v_1 = v_2$.

Exercise 1.2. Prove that if G is a graph of order p_1 and size q_1 , and H is a graph of order p_2 and size q_2 , then $G \times H$ is a graph of order $p_1 p_2$ and size $p_1 q_2 + p_2 q_1$.

1.3 Subgraphs

A **subgraph** of a graph G is a graph H whose vertex and edge sets are, respectively, subsets of the vertex and edge sets of G – i.e. $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$. A **spanning subgraph** of G is a subgraph containing all the vertices of G . A subgraph H of a graph G is an **induced subgraph** if for all vertices $u, v \in V(H)$, $uv \in E(H)$ whenever $uv \in E(G)$ – i.e. every edge of G that joins two vertices of H is present in H .

Example 1.3. Fig. 1 shows a graph G and three subgraphs H_1 , H_2 , and H_3 of G . The subgraph H_1 is neither a spanning subgraph (e.g. it does not contain v_3), nor an induced subgraph (e.g. it contains vertices v_2 and v_5 of G but not the edge v_2v_5). H_2 is a spanning subgraph and H_3 is an induced subgraph of G .

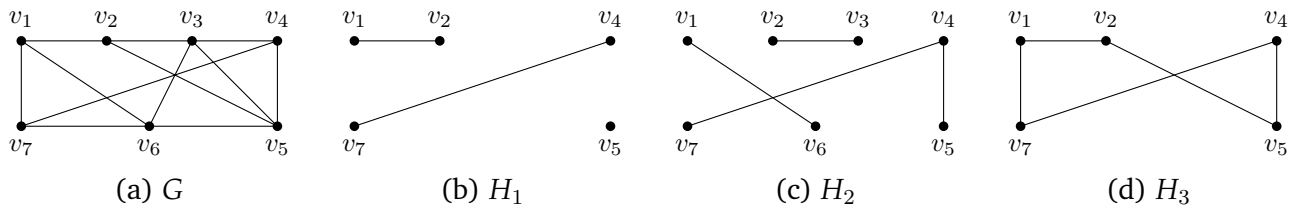


Figure 1: A graph with three of its subgraphs

A graph is **complete** if all its vertices are adjacent to one another. The complete graph of order n is denoted by K_n . As there is an edge between every pair of distinct vertices of K_n , the size of K_n is $\binom{n}{2}$. A **totally disconnected graph** has no edges.

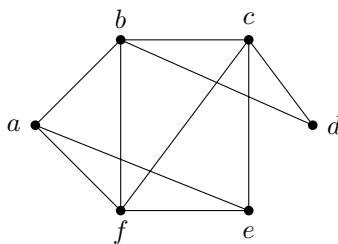
1.4 Walks, Paths, and Cycles

A **walk** in a simple graph G is a sequence of vertices $v_1, v_2, \dots, v_k$ such that $v_i \sim v_{i+1}$ for $i = 1, 2, \dots, k-1$. This is a walk **from v_1 to v_k** or **between v_1 and v_k** , having **length** $k-1$. We may also say that this is a walk of length $k-1$ starting at v_1 and ending at v_k .

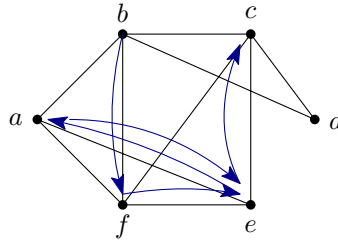
A walk in which no vertex is repeated is a **path**. Note that the length of a path is the number of edges in it.

A **closed walk** in G is a walk starting and ending at the same vertex. A closed walk in which no vertex is repeated (except for the start and end vertices being the same) is a **cycle**. The length of a cycle is also equal to the number of vertices in it. A cycle of the form $v_1, \dots, v_{k-1}, v_k$ is usually referred to as “the cycle $v_1, \dots, v_{k-1}$ ”.

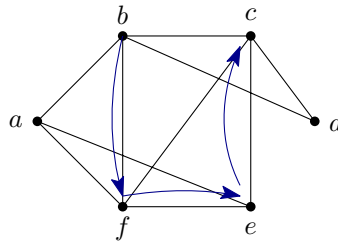
Example 1.4. Consider the graph G shown below.



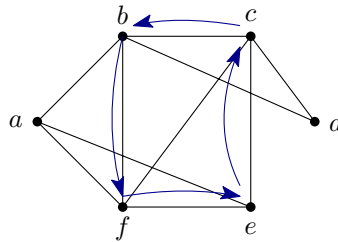
A walk of length 5 from b to c in G is b, f, e, a, e, c .



This walk is not a path, since the vertex e is repeated. An example of a path from b to c in the same graph G is b, f, e, c . The length of this path is 3.



A cycle of length 4 in G is b, f, e, c, b .



Theorem 1.5. If $\delta(G) \geq 2$, then G contains a path of length $\delta(G)$ and a cycle of length greater than $\delta(G)$.

Proof. Let $P = v_0v_1 \cdots v_k$ be a longest path in G . Then all the neighbours of v_0 must be on P (for otherwise, we would get a path longer than P). Hence, $k \geq \deg v_0 \geq \delta(G)$, so P has length at least $\delta(G)$.

Next, let $i \leq k$ be maximal such that $v_i \sim v_0$. Since all $\deg v_0$ neighbours of v_0 occur among $v_1, \dots, v_i$, we have $i \geq \deg v_0 \geq \delta(G)$. Therefore, $v_0v_1 \cdots v_iv_0$ is a cycle of length $i + 1 > \delta(G)$. $\square$

Exercise 1.3. Prove that any two longest paths in a connected graph share at least one common vertex.

1.5 Distances and Connectedness

The **distance** between two vertices u and v of a graph G is the length of a shortest path between u and v – such a shortest path between u and v is a **geodesic** from u to v . Thus, the distance between u and v in G , denoted by $d_G(u, v)$, is the length of a geodesic from u to v in G . If there is no path from u to v in G , then we set $d_G(u, v) = \infty$. When the graph G is clear from the context, we will write $d_G(u, v)$ as $d(u, v)$.

A graph is **connected** if there is a path between every two of its vertices. Otherwise, it is **disconnected**. A **(connected) component** of a graph G is a maximal connected subgraph of G . Thus, a graph G is connected if and only if it has exactly one component.

The **eccentricity** of a vertex v , denoted by $\text{ecc}(v)$, is the maximum of all distances from v to any other vertex. That is,

$$\text{ecc}(v) = \max_{u \in V(G)} d(u, v).$$

The **diameter** of G is the maximum of the eccentricities of vertices of G , and the **radius** of G is the minimum of the eccentricities of vertices of G . They are denoted by $\text{diam}(G)$ and $\text{rad}(G)$ respectively. Thus,

$$\begin{aligned} \text{diam}(G) &= \max_{v \in V(G)} \text{ecc}(v) = \max_{u, v \in V(G)} d(u, v) \\ \text{rad}(G) &= \min_{v \in V(G)} \text{ecc}(v) = \min_{v \in V(G)} \max_{u \in V(G)} d(u, v). \end{aligned}$$

The set of all vertices of G having minimum eccentricity, i.e. the set of all $v \in V(G)$ such that $\text{ecc } v = \text{rad } G$, is the **centre** of G .

Exercise 1.4. Prove that for any connected graph G , $\text{rad } G \leq \text{diam } G \leq 2 \text{ rad } G$.

1.6 Bipartite Graphs

A graph $G = (V, E)$ is **bipartite** if its vertex set V can be partitioned into two (possibly empty) subsets V_1 and V_2 such that no two vertices in V_i are adjacent, for $i = 1, 2$. That is, there exist two disjoint subsets $V_1, V_2 \subseteq V$ such that $V_1 \cup V_2 = V$, and every edge of G (if any) joins a vertex in V_1 with a vertex in V_2 . Then we say that (V_1, V_2) is a **bipartition** of G .

The following theorem characterises bipartite graphs in terms of their cycles. A cycle is **even** if its length is even and **odd** if its length is odd.

Theorem 1.6. *A graph is bipartite if and only if it contains no odd cycles.*

The **complete bipartite graph** $K_{m,n}$ is the bipartite graph with m vertices in one partite set and n vertices in the other partite set, such that every vertex of one partite set is adjacent to every vertex of the other.

1.7 Trees

A **tree** is a connected, acyclic graph. There are several well-known characterisations or alternative definitions of trees. We take the given definition as the basic one and prove its equivalence to some others. If u and v are non-adjacent vertices of a graph T , then $T + uv$ denotes the graph obtained from T by adding the edge uv .

Theorem 1.7. *If T is a graph of order p and size q , the following are equivalent:*

- (i) T is a tree.
- (ii) Any two vertices of T are joined by a unique path.
- (iii) T is connected and $p = q + 1$.
- (iv) T is acyclic and $p = q + 1$.
- (v) T is connected and for any edge e of T , $T - e$ is disconnected.
- (vi) T is acyclic and for any two non-adjacent vertices u and v of T , $T + uv$ contains a unique cycle.
- (vii) T is connected and either of order at most 2, or not complete, and for any two non-adjacent vertices u and v of T , $T + uv$ contains a unique cycle.

Exercise 1.5. A **pendant vertex** of a graph is a vertex of degree 1. Prove that every non-trivial tree contains at least two pendant vertices.

Hint: If a tree of order $p \geq 2$ had at most one vertex of degree 1, its degree sum would be at least $1 + 2(p - 1) = 2p - 1$, contradicting the Handshaking Lemma and the fact that its size is $p - 1$.

Exercise 1.6. The **centre** of a graph G is the set of all vertices of G with minimum eccentricity – i.e. the set of all vertices v of G with $\text{ecc } v = \text{rad } G$. Show that every tree has a centre consisting of either exactly one vertex or exactly two adjacent vertices.

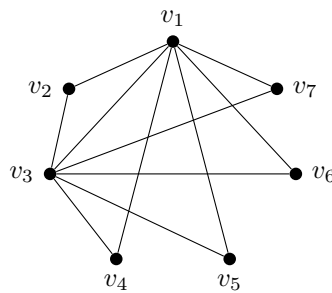
Hint: Handle trees of orders 1 and 2 separately. Deleting all pendant vertices from a tree of order at least 3 results in a smaller tree with the same centre.

1.8 Adjacency Matrices

The **adjacency matrix** of a graph G of order n , with vertex set $V = \{v_1, \dots, v_n\}$, is the $n \times n$ matrix $A = A(G)$ whose (i, j) -entry is

$$a_{ij} = \begin{cases} 1, & v_i \sim v_j \\ 0, & v_i \not\sim v_j. \end{cases}$$

Example 1.8. The adjacency matrix of the graph



is

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Observe that, as the graphs we discuss are simple graphs and therefore have no self-loops on vertices, no vertex is adjacent to itself – i.e. $a_{ii} = 0$ for all $i = 1, \dots, n$. Also, since the graphs are undirected, $v_i \sim v_j$ if and only if $v_j \sim v_i$ – i.e. $a_{ij} = a_{ji}$. Thus, we have the following observation.

Observation 1.9. The adjacency matrix of a (simple, undirected) graph is a symmetric, zero-diagonal, $(0, 1)$ -matrix.

In the i^{th} row of the adjacency matrix, for each j , the j^{th} entry is 1 if v_j is adjacent to v_i , and 0 otherwise. That is, the number of 1s in the i^{th} row is the number of vertices adjacent to v_i , or in other words, the degree of v_i . Thus, the row sums of A are the vertex degrees.

Observe that $A\mathbb{1}$ is the vector of row sums, where $\mathbb{1}$ is the vector (of suitable size) with all entries equal to 1. For instance, with the matrix A given in Example 1.8,

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 \\ 2 \\ 6 \\ 2 \\ 2 \\ 2 \\ 2 \end{bmatrix}.$$

From the Handshaking Lemma and the preceding observation, it follows that the sum of all entries of A is twice the number of edges of the graph.

Theorem 1.10. *Let A be the adjacency matrix of a graph G with vertex set $\{v_1, \dots, v_n\}$. Then the (i, j) -entry of A^m , for any positive integer m , is the number of walks of length m from v_i to v_j .*

Proof. The (i, j) -entry of A^m is

$$(A^m)_{ij} = \sum_{1 \leq k_1, \dots, k_{m-1} \leq n} a_{ik_1} a_{k_1 k_2} \cdots a_{k_{m-1} j}$$

Each term in this sum is 1 if $a_{ik_1} = \cdots = a_{k_{m-1} j} = 1$ – i.e. if $v_i \sim v_{k_1} \sim \cdots \sim v_{k_{m-1}} \sim v_j$ – and is 0 otherwise. Thus, each term counts a walk of length m from v_i to v_j . $\square$

Theorem 1.11. *Let A be the adjacency matrix of a graph G with vertex set $\{v_1, \dots, v_n\}$. Then*

- (i) $\text{tr}(A^2) = 2|E(G)|$
- (ii) $\text{tr}(A^3) = 6c_3(G)$

where $c_3(G)$ denotes the number of triangles in G .

Proof. We know that $(A^m)_{ij}$ is the number of walks of length m from v_i to v_j .

- (i) Hence, the i^{th} diagonal entry of A^2 , viz. $(A^2)_{ii}$, is the number of walks of length 2 from v_i to itself. Any walk of length 2 from v_i to itself is of the form $v_i v_j v_i$, where v_j is a vertex adjacent to v_i . Thus, the number of such walks is equal to the number of vertices adjacent to v_i , i.e. $\deg v_i$. Hence, $\text{tr}(A^2) = \sum_{i=1}^n \deg v_i = 2|E(G)|$, by Handshaking Lemma.
- (ii) Similarly, $(A^3)_{ii}$ is the number of walks of length 3 from v_i to itself. Any such walk is of the form $v_i v_j v_k v_i$, which implies that v_i, v_j , and v_k form a triangle in G . Moreover, each such triangle corresponds to two distinct walks from v_i to itself, when traversed in the two opposite directions. Thus, $(A^3)_{ii}$ is twice the number of triangles having v_i as one of its vertices. Therefore, $\text{tr}(A^3)$ is $6c_3(G)$, since each triangle contains three vertices, each of which counts the triangle twice in the sum $\text{tr}(A^3)$. $\square$

1.9 Graph Spectra

The **(adjacency) spectrum** of a graph is the spectrum (i.e. the multiset of eigenvalues) of its adjacency matrix. If the distinct eigenvalues of $A(G)$ are $\lambda_1, \dots, \lambda_k$, with respective multiplicities $m_1, \dots, m_k$, then we write

$$\text{spec } G = \begin{pmatrix} \lambda_1 & \lambda_2 & \cdots & \lambda_k \\ m_1 & m_2 & \cdots & m_k \end{pmatrix}.$$

Exercise 1.7. Show that a graph G of order n is k -regular graph if and only if the all-ones vector $\mathbb{1}_n$ is an eigenvector of $A(G)$ corresponding to the eigenvalue k .

Spectrum of the Complete Graph

The adjacency matrix of K_n is $J_n - I_n$, where J_n is the $n \times n$ all-ones matrix. Since the eigenvalues of J_n are n with multiplicity 1 and 0 with multiplicity $n - 1$, the spectrum of K_n is

$$\text{spec } K_n = \begin{pmatrix} n-1 & -1 \\ 1 & n-1 \end{pmatrix}.$$

Exercise 1.8. Verify that $\mathbb{1}_n$ is an eigenvector of $A(K_n)$ corresponding to the eigenvalue $n - 1$, and the column vectors $[1 \ 0 \ \cdots \ -1 \ \cdots \ 0]^T$, where the -1 occurs in the i^{th} position, $i = 2, \dots, n$, are independent eigenvectors corresponding to the eigenvalue -1 .

Spectrum of the Complete Bipartite Graph

The spectrum of the complete bipartite graph $K_{m,n}$ is

$$\text{spec } K_{m,n} = \begin{pmatrix} \sqrt{mn} & 0 & -\sqrt{mn} \\ 1 & m+n-2 & 1 \end{pmatrix}.$$

Spectrum of the Cycle

The eigenvalues of the cycle C_n , $n \geq 3$, are

$$\lambda_k = 2 \cos \left(\frac{2k\pi}{n} \right), \quad k = 0, \dots, n-1.$$

Spectrum of the Path

The eigenvalues of the path P_n are

$$\lambda_k = 2 \cos \left(\frac{k\pi}{n+1} \right), \quad k = 1, \dots, n.$$

1.10 Application: Centrality and Clustering

A social network can be modelled by a graph whose vertices represent people and whose edges represent relationships or interactions. Different numerical measures capture different meanings of “importance”, so identifying an influencer depends on the purpose of the analysis.

1.10.1 Centrality

Let G be a connected graph of order $n \geq 2$. The following are common measures of the centrality of a vertex v .

- (i) The **degree centrality**

$$C_D(v) = \frac{\deg(v)}{n-1}$$

measures the proportion of the network directly connected to v . A large value suggests high immediate reach.

- (ii) The **closeness centrality**

$$C_C(v) = \frac{n-1}{\sum_{\substack{u \in V(G) \\ u \neq v}} d(u, v)}$$

is large when v is only a few steps away from the rest of the network. Such a vertex can spread information efficiently.

- (iii) For distinct vertices s, t , let σ_{st} be the number of shortest s – t paths and let $\sigma_{st}(v)$ be the number of these paths that pass through v . The **betweenness centrality**

$$C_B(v) = \sum_{\{s, t\} \subseteq V(G) - \{v\}} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

measures how often v acts as a bridge between other vertices. It is particularly useful for finding brokers or potential bottlenecks.

- (iv) An **eigenvector centrality** vector is a non-negative vector $\mathbf{x}$ satisfying

$$A\mathbf{x} = \lambda\mathbf{x},$$

where A is the adjacency matrix and λ is its largest eigenvalue. Its entry x_v assigns a high score to v when v is adjacent to other high-scoring vertices.

Remark. No single centrality measure always identifies the “most influential” vertex. Degree measures direct popularity, closeness measures rapid access, betweenness measures brokerage, and eigenvector centrality values connections to already important vertices.

1.10.2 Clustering (Optional)

For a vertex v , let $N(v)$ be its set of neighbours and let $e(N(v))$ be the number of edges having both end-vertices in $N(v)$. The **local clustering coefficient** of v is

$$C(v) = \begin{cases} \frac{e(N(v))}{\binom{\deg(v)}{2}}, & \deg(v) \geq 2, \\ 0, & \deg(v) < 2. \end{cases}$$

It is the proportion of pairs of neighbours of v that are themselves adjacent. The **average clustering coefficient** of the network is

$$\bar{C}(G) = \frac{1}{n} \sum_{v \in V(G)} C(v).$$

A large local coefficient suggests that v belongs to a tightly connected community. A vertex with high centrality but relatively low clustering may instead connect different communities.

Exercise 1.9. For the graph in Example 1.8, compute the degree and closeness centralities and the local clustering coefficient of every vertex. Which vertices would be selected as influencers by these measures?

Exercise 1.10. Explain how the entries of A^2 and A^3 can help detect common neighbours and triangles in a social network.

2 Network Reliability and Routing

A communication, transport, or supply network may be represented by a graph. Vertices represent terminals, routers, or locations, while edges represent links or routes. Removing a vertex models the failure of a terminal and all its incident links; removing an edge models the failure of one link. This section studies structural points of failure, economical connected backbones, and shortest-path routing.

2.1 Cutvertices, Cutedges, and Blocks

Let $c(G)$ denote the number of components of a graph G . For $v \in V(G)$, the graph $G - v$ is obtained by deleting v and every edge incident with it. For $e \in E(G)$, the graph $G - e$ is obtained by deleting only e .

A vertex v is a **cutvertex** if $c(G - v) > c(G)$. An edge e is a **cutedge**, or a **bridge**, if $c(G - e) > c(G)$. Thus, if G is connected, a cutvertex or cutedge is one whose deletion disconnects G .

Theorem 2.1. *An edge of a graph is a cutedge if and only if it lies on no cycle.*

Proof. Let $e = uv$. If e lies on a cycle, the remaining part of that cycle is a u - v path in $G - e$. Any path that uses e can therefore be modified to avoid e , so deleting e does not increase the number of components.

Conversely, suppose that e lies on no cycle. If u and v were joined by a path in $G - e$, then that path together with e would form a cycle containing e , which is not possible. Hence u and v lie in different components of $G - e$, and thus e is a cutedge. $\square$

Theorem 2.2. *If G is connected and $v \in V(G)$, then the following are equivalent:*

- (i) v is a cutvertex of G .
- (ii) There is a partition of $V(G) - \{v\}$ into non-empty sets U and W such that every u - w path passes through v whenever $u \in U$ and $w \in W$.
- (iii) There exist vertices $u, w \in V(G) - \{v\}$ such that every u - w path passes through v .

Proof. (i) $\implies$ (ii). Since v is a cutvertex, the graph $G - v$ is disconnected, i.e. it has two or more components. Let U be the set of all the vertices in any one of the components, and let W be the set of all the remaining vertices of $G - v$. Clearly, $\{U, W\}$ is a partition of $V(G) - \{v\}$. Now, if $u \in U$ and $w \in W$, then u and w are in different components of $G - v$, which implies that any path from u to w must pass through v .

(ii) $\implies$ (iii) is obvious as the latter is a special case of the former.

(iii) $\implies$ (i). Consider the graph $G - v$. As every u - w path passes through v , none of them is present in $G - v$, and therefore $G - v$ is disconnected. Hence, v is a cutvertex of G . $\square$

The following characterisation of bridges can be proved along similar lines to that of Theorem 2.2.

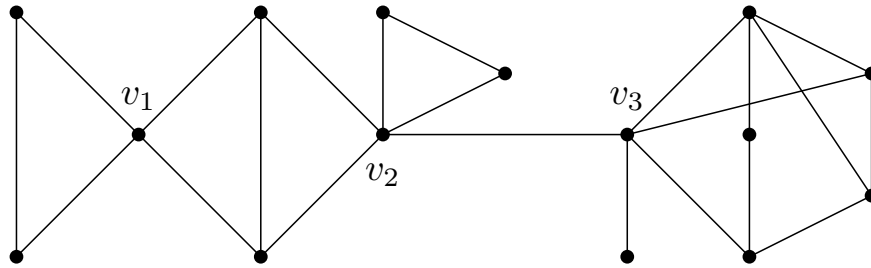
Theorem 2.3. *If G is a connected graph, and e is any edge of G , then the following are equivalent:*

- (i) e is a bridge of G .
- (ii) e is not on any cycle of G .
- (iii) There exist vertices u and w of G such that every u - w path contains e .

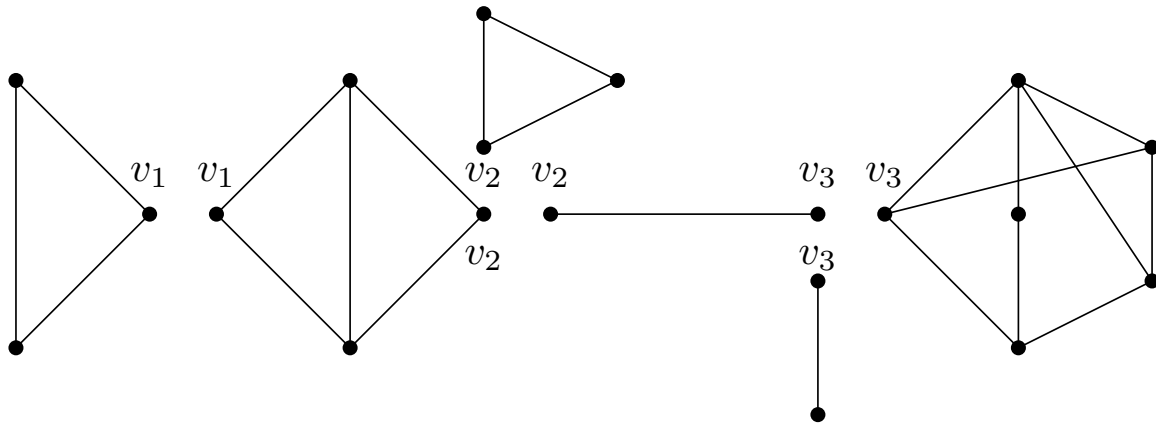
- (iv) There exists a partition of $V(G)$ into two non-empty subsets U and W such that for all $u \in U$ and $w \in W$, every u - w path contains e . $\square$

A **block** of a graph is a maximal connected subgraph that has no cutvertex. With this convention, each cutedge together with its end-vertices is a block isomorphic to K_2 , and each isolated vertex is a block isomorphic to K_1 .

Example 2.4. The graph below has six blocks and three cutvertices, namely v_1 , v_2 , and v_3 .



Its six blocks are shown separately below.



Theorem 2.5. Let G be a connected graph.

- (i) Every edge of G belongs to exactly one block.
- (ii) Any two distinct blocks have at most one vertex in common.
- (iii) A vertex belongs to at least two blocks if and only if it is a cutvertex. $\square$

Theorem 2.6. If G is a connected graph of order at least 3, then the following are equivalent:

- (i) G is a block.
- (ii) Every two vertices of G lie on a common cycle.
- (iii) Every vertex and edge of G lie on a common cycle.
- (iv) Every two edges of G lie on a common cycle. $\square$

The **block-cutvertex graph** of a connected graph G is the bipartite graph whose vertices are the blocks and cutvertices of G , with a block B adjacent to a cutvertex v precisely when $v \in V(B)$.

Theorem 2.7. The block-cutvertex graph of a connected graph is a tree. $\square$

Exercise 2.1. Prove Theorem 2.5. Also prove that the block-cutvertex graph is a tree.

Exercise 2.2. Show that every edge of a tree is a cutedge, and that the cutvertices of a non-trivial tree are exactly its non-pendant vertices.

2.1.1 Application: Network Vulnerability Assessment

Cutvertices and cutedges represent **single points of failure**: the failure of one such component separates terminals that could previously communicate. Blocks describe regions with no single vulnerable vertex, while the block-cutvertex tree records how these regions are joined.

The number of resulting components is only one measure of the severity of a failure. Suppose that G is connected and that the components of $G - v$ have orders $s_1, \dots, s_k$. The number of unordered pairs of surviving vertices separated by the failure of v is

$$P(v) = \sum_{1 \leq i < j \leq k} s_i s_j.$$

If a cutedge e separates G into components of orders s and $n - s$, then the corresponding number of separated pairs is

$$P(e) = s(n - s).$$

These quantities distinguish, for example, a bridge that isolates one terminal from a bridge that divides the network into two large pieces.

Remark. This is a structural, deterministic model. A practical reliability assessment may also need failure probabilities, capacities, repair times, and the possibility of several simultaneous failures. Moreover, a vertex with high centrality need not be a cutvertex, and a cutvertex need not have high degree.

Exercise 2.3. For the graph in Example 2.4, determine $P(v)$ for each cutvertex and $P(e)$ for each cutedge. Rank these single points of failure from most to least disruptive.

2.2 Connectivity

For $S \subseteq V(G)$, the graph $G - S$ is obtained by deleting the vertices of S and all edges incident with them. For $S \subseteq E(G)$, it is obtained by deleting the edges of S .

A set S of vertices of a connected graph G is a **separating set** or **vertex cut** of G if $G - S$ is disconnected or trivial (i.e. a graph of order 1). The **vertex connectivity** (or **connectivity**) of G , denoted by $\kappa(G)$ (or κ), is the cardinality of a minimum separating set of G – i.e. the minimum number of vertices of G whose removal results in a disconnected or trivial graph. Similarly, a **disconnecting set** of a connected graph G is a set S of edges of G such that $G - S$ is disconnected or trivial. The cardinality of a minimum disconnecting set of G is the **edge connectivity** of G , and is denoted by $\lambda(G)$ (or λ). The vertex connectivity and edge connectivity of a disconnected graph are defined to be 0.

Theorem 2.8. For any graph G , $\kappa(G) \leq \lambda(G) \leq \delta(G)$.

Proof. If $\delta(G) = 0$, then clearly, $\lambda(G) = 0$. If $\delta(G) \geq 1$, then disconnecting all the edges incident with a vertex of degree $\delta(G)$ disconnects G , and hence $\lambda(G) \leq \delta(G)$.

If G is disconnected, then $\kappa(G) = \lambda(G) = 0$. If $\lambda(G) = 1$, then G has a bridge $e = uv$. If $G \cong K_2$, deleting either vertex leaves K_1 . Otherwise, at least one of u and v is a cutvertex. Hence, in either case, $\kappa(G) = 1$.

Next, let F be a minimum disconnecting set of $\lambda \geq 2$ edges, and choose $e = uv \in F$. By the minimality of F , the graph $H = G - (F - \{e\})$ is connected, while $H - e = G - F$ is disconnected. Hence, e is a bridge of H .

For each edge of $F - \{e\}$, choose an end-vertex distinct from u and v , and let X be the set of chosen vertices. Then $|X| \leq \lambda - 1$, and all the edges of $F - \{e\}$ are absent from $G - X$.

If $G - X$ is disconnected, then X is a vertex cut. Otherwise, e remains a bridge of $G - X$; deleting a suitable end-vertex of e then leaves a disconnected or trivial graph. In either case, G has a vertex cut of cardinality at most λ , and hence $\kappa \leq \lambda$. $\square$

Theorem 2.9. *For all positive integers a , b , and c such that $a \leq b \leq c$, there exists a graph G with $\kappa(G) = a$, $\lambda(G) = b$, and $\delta(G) = c$.* $\square$

It is easy to see that $\kappa(K_n) = n - 1$ and $\kappa(K_{p,q}) = \min(p, q)$. The k -dimensional **hypercube** Q_k is the Cartesian product of k copies of K_2 , with $Q_0 = K_1$. The following theorem gives its connectivity.

Theorem 2.10. *For any $k \geq 0$, the connectivity of the hypercube Q_k is $\kappa(Q_k) = k$.*

Proof. As Q_k is k -regular, $\kappa(Q_k) \leq k$. We prove the reverse inequality by induction on k . The result is immediate for $k = 0$ and $k = 1$.

Let $k \geq 2$, suppose the result holds for Q_{k-1} , and write Q_k as two copies G_1 and G_2 of Q_{k-1} joined by a perfect matching. Suppose that $S \subseteq V(Q_k)$ and $|S| \leq k - 1$, and put $S_i = S \cap V(G_i)$ for $i = 1, 2$. We show that $Q_k - S$ is connected.

If $|S_1| \leq k - 2$ and $|S_2| \leq k - 2$, then both $G_1 - S_1$ and $G_2 - S_2$ are connected by the induction hypothesis. At least one edge of the perfect matching has neither end-vertex in S , so these two subgraphs are joined in $Q_k - S$.

Otherwise, say $|S_1| = k - 1$. Then $S_2 = \emptyset$, so G_2 is intact and connected. Every surviving vertex of G_1 has its matching neighbour in G_2 , and hence $Q_k - S$ is connected. Thus, no set of at most $k - 1$ vertices is a vertex cut, so $\kappa(Q_k) \geq k$. $\square$

Corollary 2.11. *For any $k \geq 0$, the edge connectivity of the hypercube Q_k is $\lambda(Q_k) = k$.*

Proof. From Theorem 2.8, $k = \kappa(Q_k) \leq \lambda(Q_k) \leq \delta(Q_k) = k$. $\square$

Theorem 2.12. *If G is 3-regular, then $\kappa(G) = \lambda(G)$.*

Proof. If G is disconnected, then $\kappa(G) = \lambda(G) = 0$. If $G \cong K_4$, then $\kappa(G) = \lambda(G) = 3$. We may therefore suppose that G is connected and not complete.

Let S be a minimum vertex cut of G . We know that $\lambda(G) \geq \kappa(G) = |S|$. The graph $G - S$ has at least two components, and the minimality of S implies that every vertex of S has a neighbour in every component of $G - S$.

If $G - S$ has three components, then every vertex of S has exactly one neighbour in each component. The set of edges joining S to any one component is therefore a disconnecting set of size $|S|$.

Now suppose that $G - S$ has two components, H_1 and H_2 . A vertex of S with a neighbour in S has exactly one neighbour in each H_i ; delete its edge to H_1 . A vertex of S with no neighbour in S has one neighbour in one of the H_i and two in the other; delete its edge to the component in which it has only one neighbour. Exactly one edge has now been deleted for each vertex of S , and no remaining path joins H_1 to H_2 . Thus, $\lambda(G) \leq |S| = \kappa(G)$, and equality follows. $\square$

2.3 Minimum Spanning Trees

A **tree** is a connected graph containing no cycles. A **spanning tree** of a connected graph G is a spanning subgraph of G that is a tree. If every edge e of G has a real weight $w(e)$, the weight of a subgraph H is

$$w(H) = \sum_{e \in E(H)} w(e).$$

A **minimum spanning tree** of G is a spanning tree having minimum weight among all spanning trees of G .

For a non-empty proper subset S of $V(G)$, the edges with one end-vertex in S and the other in $V(G) - S$ form an **edge cut** of G .

Theorem 2.13 (Cut Property). *Let e be an edge of minimum weight in an edge cut of a connected weighted graph G . Then some minimum spanning tree contains e . If e is the unique minimum-weight edge in the cut, every minimum spanning tree contains e .*

Proof. Let the cut be determined by $S \subset V(G)$, and let T be a minimum spanning tree. There is nothing to prove if $e \in E(T)$. Otherwise, adding e to T creates a unique cycle. As e crosses the cut, this cycle contains another edge f that crosses the cut. Since e has minimum weight in the cut, $w(e) \leq w(f)$. Replacing f by e therefore gives a spanning tree of weight at most $w(T)$, and hence another minimum spanning tree. If e is the unique minimum-weight edge in the cut, then $w(e) < w(f)$; consequently, a minimum spanning tree omitting e would not be minimum. $\square$

Prim's algorithm grows a tree from an arbitrary starting vertex. At each step, it chooses a cheapest edge joining a vertex already reached to a vertex not yet reached.

Algorithm 1 Prim's algorithm

Input: Connected weighted graph G and starting vertex r

```

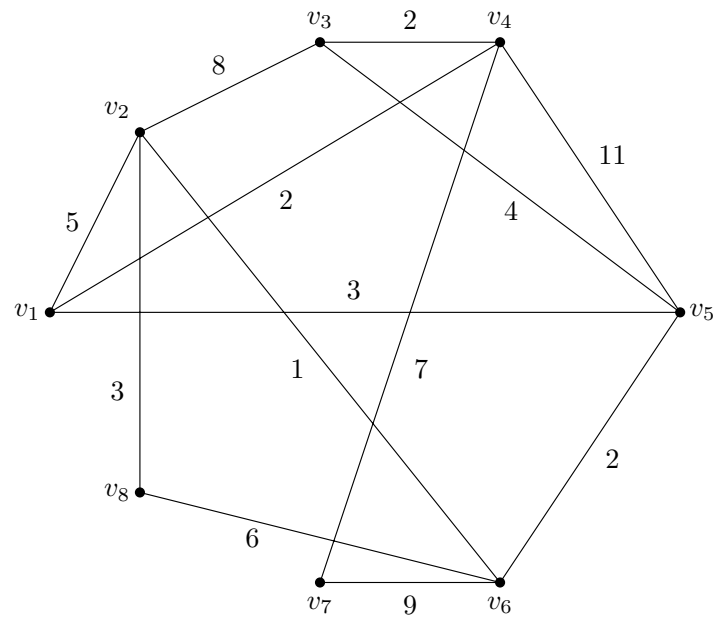
1:  $S \leftarrow \{r\}, F \leftarrow \emptyset$ 
2: while  $S \neq V(G)$  do
3:   Choose a minimum-weight edge  $xy$  with  $x \in S$  and  $y \notin S$ 
4:    $F \leftarrow F \cup \{xy\}$ 
5:    $S \leftarrow S \cup \{y\}$ 
6: end while
7: return  $T = (V(G), F)$ 
```

Theorem 2.14. *Prim's algorithm returns a minimum spanning tree.*

Proof. We maintain the invariant that some minimum spanning tree contains every edge in F . This is true initially. Suppose that a minimum spanning tree T contains F , and let e be the next edge chosen across the cut determined by S . If $e \in E(T)$, the invariant is preserved. Otherwise, adding e to T creates a unique cycle containing another edge f that crosses the same cut. No edge of F crosses this cut, and hence $f \notin F$. Since e has minimum weight in the cut, $w(e) \leq w(f)$, so $T - f + e$ is a minimum spanning tree containing $F \cup \{e\}$. Thus, the invariant is preserved.

Each iteration adds one new vertex and one edge without creating a cycle. On termination, F is itself a spanning tree; since some minimum spanning tree contains all its edges, F is a minimum spanning tree. $\square$

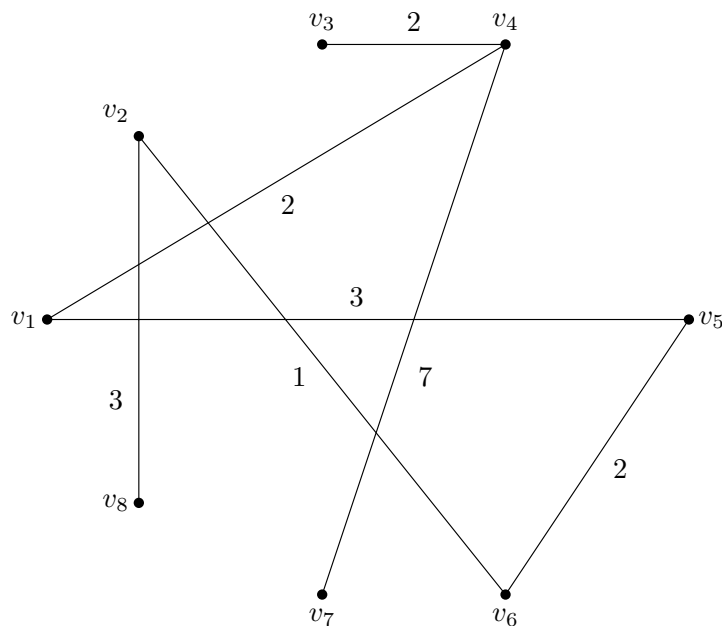
Example 2.15. Apply Prim's algorithm to the graph below, taking v_1 as the starting vertex.



One possible sequence of choices is shown in the table.

Step	Edge added	Vertex added	Total weight
1	—	v_1	0
2	v_1v_4	v_4	2
3	v_4v_3	v_3	4
4	v_1v_5	v_5	7
5	v_5v_6	v_6	9
6	v_6v_2	v_2	10
7	v_2v_8	v_8	13
8	v_4v_7	v_7	20

The resulting minimum spanning tree has weight 20.



Remark. A minimum spanning tree is a cheapest connected backbone, but it has no redundant edge: every edge of a tree is a cutedge. Cost minimisation and resilience are therefore different design objectives.

Exercise 2.4. Explain why Prim's algorithm can produce different minimum spanning trees when edge weights are repeated. Show that if all edge weights are distinct, the minimum spanning tree is unique.

2.4 All-Pairs Shortest Paths

Let G be a weighted directed graph with vertices $v_1, \dots, v_n$. Its **weighted adjacency matrix** is $W = (w_{ij})$, where

$$w_{ij} = \begin{cases} 0, & i = j, \\ \text{weight of the edge from } v_i \text{ to } v_j, & (v_i, v_j) \in E(G), \\ \infty, & (v_i, v_j) \notin E(G). \end{cases}$$

We assume that G has no negative-weight cycle. Edge weights may be negative. In the absence of a negative-weight cycle, a minimum-weight walk can always be reduced to a path. If a negative-weight cycle is reachable from v_i and can reach v_j , however, the weights of v_i - v_j walks are unbounded below, so there is no finite shortest-path distance from v_i to v_j .

For $k = 0, 1, \dots, n$, let $d_{ij}^{(k)}$ be the minimum weight of a v_i - v_j path whose internal vertices belong to $\{v_1, \dots, v_k\}$. Initially, $D^{(0)} = W$. For $k \geq 1$, a shortest admissible path either avoids v_k , or passes through v_k . Therefore,

$$d_{ij}^{(k)} = \min \left\{ d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right\}.$$

After the final stage, $d_{ij}^{(n)}$ is the distance from v_i to v_j .

Algorithm 2 Floyd–Warshall algorithm

Input: Weighted adjacency matrix W of order n

```

1:  $D \leftarrow W$ 
2: for  $k = 1, \dots, n$  do
3:   for  $i = 1, \dots, n$  do
4:     for  $j = 1, \dots, n$  do
5:        $d_{ij} \leftarrow \min\{d_{ij}, d_{ik} + d_{kj}\}$ 
6:     end for
7:   end for
8: end for
9: return  $D$ 
```

Theorem 2.16. If G has no negative-weight cycle, the matrix returned by the Floyd–Warshall algorithm is its all-pairs distance matrix.

Proof. After iteration k , the entry d_{ij} equals $d_{ij}^{(k)}$. This follows by induction on k from the recurrence above. At $k = n$, every vertex may be used internally, so each entry is the weight of a shortest path between the corresponding vertices. $\square$

The algorithm performs n^3 comparisons and uses an $n \times n$ matrix. Hence, its running time is $O(n^3)$ and its storage requirement is $O(n^2)$. If some diagonal entry is negative after the algorithm terminates, the graph contains a negative-weight cycle.

2.5 Application: Routing Tables

A distance matrix gives route costs but not the routes themselves. To construct routing tables, maintain a **next-hop matrix** N . Initially, set $N_{ij} = j$ when (v_i, v_j) is an edge, and leave N_{ij} undefined when $i \neq j$ and there is no such edge. Whenever the Floyd–Warshall update through v_k strictly improves d_{ij} , also set

$$N_{ij} \leftarrow N_{ik}.$$

Then N_{ij} is the first vertex to visit on a shortest route from v_i to v_j . Repeatedly consulting the appropriate row reconstructs the complete route.

Example 2.17. Consider the weighted adjacency matrix

$$W = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix}.$$

Floyd–Warshall gives the distance and next-hop matrices

$$D = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix},$$

$$N = \begin{bmatrix} - & 2 & 2 & 2 \\ 3 & - & 3 & 3 \\ 4 & 4 & - & 4 \\ 1 & 1 & 1 & - \end{bmatrix}.$$

For example, the entry $N_{14} = 2$ directs traffic from v_1 first to v_2 . The successive next hops give the shortest route v_1, v_2, v_3, v_4 , of total weight 6.

Exercise 2.5. Apply Floyd–Warshall to the matrix in the preceding example, recording the matrix after each value of k . Verify the final distance and next-hop matrices.

Exercise 2.6. Suppose the weight of the edge from v_3 to v_4 in the preceding example increases from 1 to 6. Recompute the routing table and identify every source–destination pair whose shortest route changes.